

Lab 8: Spark Structured Streaming

Z5008 Big Data Lab — MTech Data Science & AI

Dr. Innocent Nyalala
`innocent@iitmz.ac.in`

School of Engineering and Science
IIT Madras Zanzibar

Monday, 4 May 2026

 Real-time data pipelines for East Africa

Session Agenda

Theory (60 min)

- 1 Streaming vs. batch: the unbounded table model
- 2 Trigger modes & output modes
- 3 Sources: Kafka, File, Rate, Socket
- 4 Watermarks & late-data handling
- 5 Window aggregations (tumbling, sliding)
- 6 Stateful operations
- 7 Sinks: console, file, Kafka, foreachBatch
- 8 Checkpointing & exactly-once
- 9 Delta Lake streaming + MERGE
- 10 Stream-stream joins
- 11 Query management & monitoring
- 12 Performance tuning

Lab (90 min)

- 1 Environment setup & Kafka connectivity
- 2 Read & parse Kafka events
- 3 Watermark + window aggregation
- 4 Write to Delta Lake (foreachBatch MERGE)
- 5 Monitor Spark UI Streaming tab
- 6 Assignment briefing

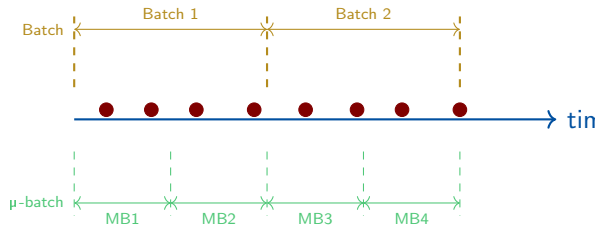
Assignment

Due: **Sunday 10 May 2026, 23:59 EAT**
100 marks — 5 tasks

Why Real-Time Streaming?

East African Use Cases

- 🇰🇪 M-Pesa / Airtel Money fraud detection
- ❤️ IoT health monitoring (Zanzibar clinics)
- 📶 Telecom network anomaly detection
- 🌱 Precision agriculture: real-time weather
- 🚢 Port logistics: vessel AIS streams



Batch vs. Streaming

Batch	Streaming
Hours/minutes latency	Sub-second latency
Bounded dataset	Unbounded dataset
Process once	Continuous processing

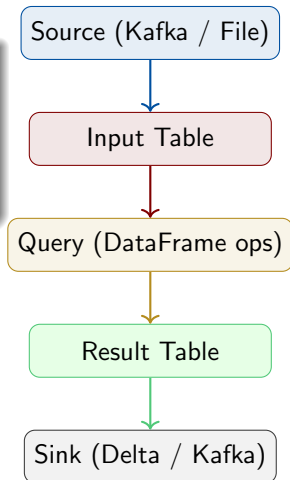
The Unbounded Table Abstraction

Core Concept

Spark Structured Streaming treats a live data stream as an **unbounded table** that grows continuously. Each trigger appends new rows; queries run on the complete logical table.

Mental Model:

- 1 Events arrive from a source (Kafka, files, ...)
- 2 Spark appends them to the *input table*
- 3 Your query produces a *result table*
- 4 The result table is written to a *sink*
- 5 Only the *delta* (changed rows) is output each trigger



Trigger Modes

Trigger	When to use
Default (ASAP)	Low-latency production
ProcessingTime	Rate-limited pipelines
Once	One-shot, then stop
AvailableNow	Catch-up then stop
Continuous	Near-true streaming

Continuous Mode

Uses an asynchronous checkpoint protocol. Latency down to ~ 1 ms but limited operator support. Use `processingTime` triggers for most workloads.

```
1 # Default: process ASAP
2 q = df.writeStream \
3   .format("console") \
4   .start()
5
6 # Fixed interval
7 q = df.writeStream \
8   .trigger(processingTime=
9             "30 seconds") \
10  .format("console").start()
11
12 # Run once then stop
13 q = df.writeStream \
14   .trigger(once=True) \
15   .format("parquet").start()
16
17 # Continuous (experimental)
18 q = df.writeStream \
19   .trigger(continuous=
20             "1 second") \
21   .format("console").start()
22
```

Output Modes

Three Output Modes

- Append** Only *new* rows added to the result table are written to the sink. Valid for: no aggregation; windowed agg *with* watermark.
- Update** Only rows that *changed* this trigger are written. Valid for any aggregation; console.
- Complete** The *entire* result table is rewritten each trigger. Requires all state in memory. Valid only for aggregations.

Mode vs. Query Type

Mode	Allowed with
Append	No agg; windowed agg + watermark
Update	Any aggregation; no sorting
Complete	Aggregation (all state retained)

Common Mistakes

Using complete mode without aggregation → error.

Using append mode with agg without watermark → error.

Streaming Sources

Kafka — production source

```
1 raw = spark.readStream \  
2   .format("kafka") \  
3   .option("kafka.bootstrap.servers", \  
4         "kafka:9092") \  
5   .option("subscribe", \  
6         "transactions") \  
7   .option("startingOffsets", \  
8         "earliest") \  
9   .option("maxOffsetsPerTrigger", \  
10        5000) \  
11   .load() \  
12
```

Rate — testing only

```
1 test = spark.readStream \  
2   .format("rate") \  
3   .option("rowsPerSecond", 100) \  
4   .load() \  
5
```

File source (parquet / json)

```
1 files = spark.readStream \  
2   .format("parquet") \  
3   .schema(my_schema) \  
4   .option("path", \  
5         "s3a://bucket/landing/") \  
6   .load() \  
7
```

Socket — debugging only

```
1 sock = spark.readStream \  
2   .format("socket") \  
3   .option("host", "localhost") \  
4   .option("port", 9999) \  
5   .load() \  
6
```

Note

Kafka Source: Schema and Key Options

Kafka DataFrame Schema

- key: BINARY
- value: BINARY ← your payload
- topic: STRING
- partition: INT
- offset: LONG
- timestamp: TIMESTAMP
- timestampType: INT

Important Options

`subscribe` Comma-separated topic list

`subscribePattern` Regex pattern

`startingOffsets` earliest/latest/JSON map

`failOnDataLoss` Set false for dev

`maxOffsetsPerTrigger` Back-pressure control

Parsing Kafka Value → Struct

```
1 from pyspark.sql.functions import \
2     col, from_json
3 from pyspark.sql.types import *
4
5 schema = StructType([
6     StructField("txn_id",
7                 StringType()),
8     StructField("amount",
9                 DoubleType()),
10    StructField("country",
11                StringType()),
12    StructField("event_time",
13                TimestampType()),
14 ])
15
16 parsed = (raw
17           .select(col("value")
18                   .cast("STRING")
19                   .alias("raw_json"))
20           .select(from_json(
```

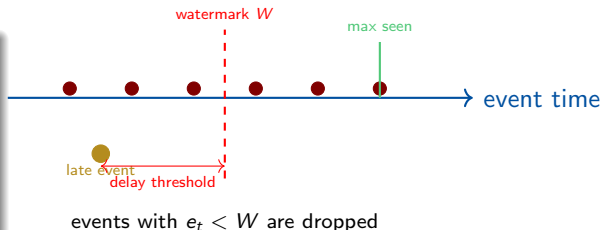

Watermarks and Late Data

What is a Watermark?

A watermark tells Spark how late an event can arrive and still be included in window computations.

$$W = \max(e_t \text{ seen}) - \text{delay threshold}$$

Events with $e_t < W$ are **dropped**.



Why it matters

- Without watermarks, state grows unboundedly
- Watermarks bound state size; Spark cleans up old windows
- Required for append mode with aggregations
- Required for stream-stream joins

Applying Watermarks and Window Aggregations

```
1 from pyspark.sql.functions import col, window, count, sum as _sum
2
3 # Apply 2-minute watermark on event_time
4 with_wm = parsed.withWatermark("event_time", "2 minutes")
5
6 # Tumbling window: non-overlapping, fixed duration
7 tumbling = (
8     with_wm
9     .groupBy(window(col("event_time"), "5 minutes"), col("country"))
10    .agg(
11        count("txn_id").alias("txn_count"),
12        _sum("amount").alias("total_amount"),
13    )
14 )
15
16 # Sliding window: overlapping (slide < window duration)
17 sliding = (
18     with_wm
19     .groupBy(
20         window(col("event_time"), "10 minutes", "2 minutes"),
21         col("country"),
22     )
```

Stateful Streaming Operations

Built-in stateful ops

- `groupBy().agg()` — per-key aggregation
- `groupBy(window,...).agg()` — windowed agg
- `dropDuplicates()` — deduplication
- Stream-stream joins

Custom state: `mapGroupsWithState`

```
def update_fn(key, vals, state):  
    prev = state.getOrElse() or 0  
    total = prev + sum(  
        v.amount for v in vals  
    )  
    state.update(total)  
    return (key, total)
```

```
result = parsed \  
    .groupBy("country") \  
    .mapGroupsWithState(  
        update_fn, state = Map.empty, as = List.empty
```

`flatMapGroupsWithState`

```
# Emits 0..N rows per group/trigger  
def detect_fraud(key, vals, state):  
    for v in vals:  
        if v.amount > 10_000:  
            yield Row(  
                alert_country=key,  
                amount=v.amount)  
            state.update(new_state)  
  
alerts = parsed \  
    .groupBy("country") \  
    .flatMapGroupsWithState(  
        detect_fraud, ...)
```

State Timeout

Use `GroupStateTimeout.ProcessingTime`
`Timeout` or `EventTimeTimeout` to expire stale

Streaming Sinks

Console (debug only)

```
q = df.writeStream \
    .outputMode("update") \
    .format("console") \
    .option("truncate", False) \
    .start()
```

File / Delta

```
q = df.writeStream \
    .outputMode("append") \
    .format("delta") \
    .option("checkpointLocation",
            "s3a://bucket/ckpt/") \
    .option("path",
            "s3a://bucket/delta/txns/") \
    .start()
```

Kafka sink

```
q = df \
    .select(to_json(struct("*"))
            .alias("value")) \
    .writeStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers",
            "kafka:9092") \
    .option("topic", "results") \
    .option("checkpointLocation",
            "s3a://bucket/ckpt/k/") \
    .start()
```

foreachBatch

```
def process_batch(df, epoch_id):
    df.write.format("jdbc").save()

q = agg_df.writeStream \
    .foreachBatch(process_batch) \
```

foreachBatch: The Power Pattern

Why foreachBatch?

- Treat each micro-batch as a static DataFrame
- Apply any batch operation: MERGE, JDBC, REST API
- Write to multiple sinks in one function
- Use epoch_id for idempotent exactly-once writes

MERGE Pattern (Upsert)

Delta Lake MERGE upserts streaming results into a target table without duplicates, enabling real-time materialized views.

```
1 from delta.tables import DeltaTable
2
3 def upsert_to_delta(batch_df, epoch_id):
4     target = DeltaTable.forPath(
5         spark,
6         "s3a://bucket/delta/summary/")
7
8     (target.alias("t")
9      .merge(
10         batch_df.alias("s"),
11         "t.country = s.country AND "
12         "t.window_start = s."
13         window_start")
14        .whenMatchedUpdateAll()
15        .whenNotMatchedInsertAll()
16        .execute())
17
18 query = agg_df.writeStream \
19     .outputMode("update") \
20     .foreachBatch(upsert_to_delta) \
21     .option("checkpointLocation",
22            "s3a://bucket/ckpt/merge/") \
```

Checkpointing and Fault Tolerance

What checkpointing stores

- 1 **Offsets** — which Kafka offsets have been read
- 2 **State** — aggregation state (window totals, etc.)
- 3 **Commits** — which micro-batches are complete

Mandatory

Checkpointing is **required** for fault tolerance and exactly-once delivery. Without it, a restart replays from scratch.

Checkpoint directory layout

- `offsets/` — per-batch source offsets
- `commits/` — completed batch IDs

```
1 CKPT = "s3a://datalake/checkpoints"
2
3 query = agg_df.writeStream \
4     .outputMode("append") \
5     .format("delta") \
6     .option("checkpointLocation",
7         f"{CKPT}/window_agg/") \
8     .option("path",
9         "s3a://datalake/delta/agg/") \
10    .trigger(
11        processingTime="30 seconds") \
12    .start()
13
```

Exactly-once requires:

- 1 Replayable source (Kafka, file)
- 2 Idempotent or transactional sink (Delta, Kafka 0.11+)
- 3 Checkpoint enabled

Delta Lake as Streaming Source and Sink

Read stream from Delta

```
1 # Delta table as streaming source
2 delta_stream = (
3     spark.readStream
4         .format("delta")
5         .option("maxFilesPerTrigger", 1)
6         .load("s3a://bucket/delta/raw/")
7 )
8
```

Write stream to Delta (append)

```
1 q = parsed.writeStream \
2     .format("delta") \
3     .outputMode("append") \
4     .option("checkpointLocation",
5         "s3a://bucket/ckpt/raw/") \
6     .option("path",
7         "s3a://bucket/delta/raw/") \
8     .start()
9
```

Inspect Delta history

```
1 from delta.tables import DeltaTable
2
3 dt = DeltaTable.forPath(
4     spark,
5     "s3a://bucket/delta/raw/")
6
7 # Shows streaming commits
8 dt.history(5).show(truncate=False)
9 # operation: STREAMING UPDATE
10 # operationMetrics:
11 #   numAddedFiles, numOutputRows
12
```

Delta + Streaming Advantages

ACID transactions, schema enforcement, time travel, efficient MERGE (upsert), and full audit history — all integrated with structured streaming.

Stream–Stream Joins

Challenge

Both sides are unbounded — Spark must buffer rows until matching rows arrive. Watermarks bound the buffer size.

Requirements

- Both streams must have watermarks
- Join condition must include event-time bounds
- Supported: inner, left outer, right outer (with restrictions)
- Full outer: experimental

```
1 txns = kafka_stream1 \  
2   .withWatermark("txn_time",  
3                 "2 minutes")  
4  
5 kyc = kafka_stream2 \  
6   .withWatermark("kyc_time",  
7                 "5 minutes")  
8  
9 from pyspark.sql.functions import expr  
10  
11 joined = txns.join(  
12     kyc,  
13     expr("""  
14         txns.user_id = kyc.user_id  
15         AND txn_time BETWEEN  
16             kyc_time - interval 5 minutes  
17             AND  
18             kyc_time + interval 2 minutes  
19         """),  
20     "inner"  
21 )
```


StreamingQuery Management

```
1 # Start a query
2 q = df.writeStream \
3     .format("console") \
4     .start()
5
6 # Check status
7 print(q.status)
8 # {'message': 'Waiting ...',
9 #   'isDataAvailable': True,
10 #   'isTriggerActive': False}
11
12 # Last batch progress
13 import json
14 print(json.dumps(
15     q.lastProgress, indent=2))
16 # numInputRows,
17 # inputRowsPerSecond,
18 # processedRowsPerSecond,
19 # durationMs, sources, sink
```

```
1 # Block until termination
2 q.awaitTermination()
3
4 # Block with timeout
5 q.awaitTermination(timeout=60)
6
7 # Graceful stop
8 q.stop()
9
10 # List all active queries
11 active = spark.streams.active
12
13 # Add progress listener
14 from pyspark.sql.streaming import \
15     StreamingQueryListener
16
17 class MyListener(StreamingQueryListener)
18 :
19     def onQueryProgress(self, event):
20         rows = event.progress.numInputRows
21         print(f"Rows in batch: {rows}")
```

Monitoring: Spark UI Streaming Tab

Access Spark UI

- Spark UI: `http://localhost:4040`
- Streaming tab shows per-query metrics in real time

Key metrics to watch

Input Rate Events per second arriving from source

Processing Rate Events per second processed by Spark

Batch Duration Time to complete each micro-batch

State Rows Rows held in stateful operators

Trigger Gap Time between trigger start times

```
Streaming Query:  transactions
Status:  ACTIVE
Input Rate:  1,240 rows/s
Processing Rate:  1,850 rows/s
Batch Duration:  avg 420 ms
Aggregated Since Start:
Total Batches:  47
Total Rows Processed:  58,280
Avg Input Rate:  1,100/s
Last Batch (ID: 47):
Start:  14:32:05
Duration:  398 ms
Input Rows:  520
State Rows:  142
```

Performance Tuning

1. Back-pressure

```
.option("maxOffsetsPerTrigger",  
        10000)
```

2. Trigger interval

```
.trigger(processingTime=  
        "1 minute")
```

3. Shuffle partitions

```
spark.conf.set(  
    "spark.sql.shuffle.partitions",  
    "8")  # match Kafka partition count
```

4. RocksDB state backend

```
spark.conf.set(  
    "spark.sql.shuffle.partitions",  
    "8")
```

5. Kafka consumer tuning

```
.option("kafka.fetch.min.bytes",  
        "65536")  
.option("kafka.fetch.max.wait.ms",  
        "500")
```

General Guidelines

- Use AvailableNow for catch-up processing
- Minimize UDFs in hot path — use native functions
- Cache reference DataFrames used in foreachBatch
- Compact Delta checkpoints periodically
- Use columnar storage for wide state schemas
- Monitor GC pressure on executors via JVM metrics

SparkSession: Full Lab Configuration

```
1 from pyspark.sql import SparkSession
2
3 spark = (
4     SparkSession.builder
5     .appName("Lab8_StructuredStreaming")
6     .master("spark://spark-master:7077")
7     # Kafka + Delta packages
8     .config("spark.jars.packages",
9            "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0,"
10            "io.delta:delta-core_2.12:2.4.0")
11     .config("spark.sql.extensions",
12            "io.delta.sql.DeltaSparkSessionExtension")
13     .config("spark.sql.catalog.spark_catalog",
14            "org.apache.spark.sql.delta.catalog.DeltaCatalog")
15     # MinIO / S3A
16     .config("spark.hadoop.fs.s3a.endpoint", "http://minio:9000")
17     .config("spark.hadoop.fs.s3a.access.key", "minioadmin")
18     .config("spark.hadoop.fs.s3a.secret.key", "minioadmin")
19     .config("spark.hadoop.fs.s3a.path.style.access", "true")
20     .config("spark.hadoop.fs.s3a.impl",
21            "org.apache.hadoop.fs.s3a.S3AFileSystem")
22     # Performance
```

End-to-End Pipeline — Part 1: Read & Parse

```
1 from pyspark.sql.functions import col, from_json, window, count, sum as _sum
2 from pyspark.sql.types import *
3
4 # 1. Schema for mobile-money transaction events
5 TXN_SCHEMA = StructType([
6     StructField("txn_id",      StringType(),      True),
7     StructField("user_id",      StringType(),      True),
8     StructField("amount",       DoubleType(),       True),
9     StructField("country",      StringType(),       True),
10    StructField("event_time",    TimestampType(),   True),
11 ])
12
13 # 2. Read raw bytes from Kafka
14 raw = (spark.readStream.format("kafka")
15     .option("kafka.bootstrap.servers", "kafka:9092")
16     .option("subscribe", "mobile_money")
17     .option("startingOffsets", "latest")
18     .option("maxOffsetsPerTrigger", 2000)
19     .load())
20
21 # 3. Decode BINARY value -> STRING -> Struct
22 parsed = (raw
```

End-to-End Pipeline — Part 2: Aggregate & MERGE

```
1  # 4. Watermark + 5-minute tumbling window aggregation
2  agg = (
3      parsed
4      .withWatermark("event_time", "2 minutes")
5      .groupBy(window(col("event_time"), "5 minutes"), col("country"))
6      .agg(
7          count("txn_id").alias("txn_count"),
8          _sum("amount").alias("total_amount"),
9      )
10     .select(
11         col("window.start").alias("window_start"),
12         col("window.end").alias("window_end"),
13         col("country"), col("txn_count"), col("total_amount"),
14     )
15 )
16
17 TARGET = "s3a://datalake/delta/txn_summary/"
18 CKPT    = "s3a://datalake/ckpt/txn_summary/"
19
20 from delta.tables import DeltaTable
21
22 def upsert_batch(batch_df, epoch_id):
```

End-to-End Pipeline — Part 3: Start & Monitor

```
1 import time, json
2
3 # 5. Start the streaming query
4 query = (agg.writeStream
5           .outputMode("update")
6           .foreachBatch(upsert_batch)
7           .option("checkpointLocation", CKPT)
8           .trigger(processingTime="30 seconds")
9           .start())
10
11 print(f"Query ID   : {query.id}")
12 print(f"Status     : {query.status}")
13
14 # 6. Monitor for five triggers
15 for i in range(5):
16     time.sleep(35)
17     if query.lastProgress:
18         prog = query.lastProgress
19         print(f"Batch {i+1}: "
20               f"inputRows={prog['numInputRows']}, "
21               f"durationMs={prog['durationMs']}['triggerExecution']}")
22
```

Common Errors and Fixes

Error	Cause	Fix
Queries with streaming sources must be executed with writeStream	Called show() on streaming DF	Replace with writeStream
Append output mode not supported	Agg without watermark + append mode	Add withWatermark()
Cannot resolve column "value"	Forgot .cast("STRING")	Cast Kafka binary value first
DeltaTable not found	Target Delta path does not exist	Create with first batch .format("delta").save()
Checkpoint location required	No checkpoint option set	Add .option("checkpointLocation", ...)
Connection refused kafka:9092	Kafka container not ready	Wait for healthcheck; set failOnDataLoss=false

Lab Exercise Checklist

Step-by-Step

- 1 Verify Docker services running
- 2 Build SparkSession with Kafka + Delta packages
- 3 Start Kafka producer for `mobile_money` topic
- 4 Read stream, decode Kafka binary value
- 5 Console sink — inspect raw parsed events
- 6 Add watermark + 5-min tumbling window aggregation
- 7 Write aggregated stream to Delta Lake
- 8 Implement `foreachBatch` MERGE (upsert)
- 9 Inspect Delta DESCRIBE HISTORY
- 10 Monitor Spark UI Streaming tab at port 4040
- 11 Stop query gracefully with `query.stop()`

Service Endpoints

`http://localhost:8888` — JupyterLab
`http://localhost:8080` — Spark Master
`http://localhost:4040` — Spark UI
`http://localhost:9001` — MinIO Console
`kafka:9092` — Kafka bootstrap
`spark://spark-master:7077` — Spark master

Assignment Due

Sunday, 10 May 2026, 23:59 EAT

Submit `.ipynb` + PDF via Moodle.

100 marks — 5 tasks.

Lab 8 Summary

Concepts Covered

- Unbounded table model — continuous DataFrames
- Triggers: default, ProcessingTime, Once, AvailableNow, Continuous
- Sources: Kafka, File, Rate, Socket
- Kafka parsing: BINARY → STRING → from_json
- Watermarks for late-data handling and state cleanup
- Tumbling and sliding window aggregations
- Stateful ops: mapGroupsWithState, flatMapGroupsWithState
- Output modes: append, update, complete
- Sinks: console, file, Kafka, foreachBatch

Next Week — Lab 9

MLlib at Scale: distributed machine learning with Spark.

Feature engineering pipelines, classifiers, cross-validation, model persistence to S3A.

Further Reading

- Spark Structured Streaming Guide
<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- Delta Lake Streaming
<https://docs.delta.io/latest/delta-streaming.html>
- Kafka + Spark Integration